Name: Satorre, Lance Timothy
GitHub Link:
https://github.com/Lancesatorre/IntProgActivities/tree/main/Lab5-Activity_SATORRE/typescript-crud-api

TypeScript CRUD API

1. Architecture Overview

The API uses a layered architecture with clear separation of concerns. Each layer has a single responsibility and communicates only with adjacent layers.

Layer Diagram

```
┌──────────────────────────────────────┐
│  Express Server  (server.ts)         │   Port 4000
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│  Routes  (users.controller.ts)       │   POST / GET / PUT / DELETE
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│  Validation & Middleware             │   validateRequest, errorHandler
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│  Business Logic  (user.service.ts)   │   CRUD operations, password hash
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│  ORM Layer  (user.model.ts)          │   Sequelize models
└──────────────────────────────────────┘
              │
              ▼
┌──────────────────────────────────────┐
│  MySQL Database                      │   typescript_crud_api schema
└──────────────────────────────────────┘
```

Key Design Principles

- **Separation of Concerns** – each layer has one clearly defined responsibility
- **Type Safety** – strong TypeScript typing from route handler to database
- **Centralized Error Handling** – all errors routed through a single middleware
- **Validation First** – input is validated before business logic executes
- **DRY** – shared utilities live in _helpers/ and _middleware/

2. Type System & Interfaces

UserAttributes

Represents the full database record shape:

```typescript
export interface UserAttributes {
    id: number;
    email: string;
    passwordHash: string;
    title: string;
    firstName: string;
    lastName: string;
    role: string;
    createdAt?: Date;
    updatedAt?: Date;
}
```

Field Reference

Field	Type	Notes
id	number	Auto-increment primary key
email	string	Unique — used as login identifier
passwordHash	string	bcrypt hash, never stored in plaintext
title	string	Honorific: Mr, Ms, Dr, etc.
firstName	string	
lastName	string	
role	string	See Role enum
createdAt	Date?	Auto-managed by Sequelize
updatedAt	Date?	Auto-managed by Sequelize

UserCreationAttributes

```
export interface UserCreationAttributes extends Optional<UserAttributes, 'id' | 'createdAt' | 'updatedAt'> {}
```

Excludes auto-generated fields so API consumers do not need to supply them.

Role Enum

```
export enum Role{
    Admin = 'Admin',
    User = 'User'
}
```

Using the enum prevents invalid strings and enables compile-time validation.

3. Database Setup

Configuration

```
escript-crud-api > {} config.json > ...
{
    "database": {
        "host": "localhost",
        "port": 3306,
        "user": "root",
        "password": "",
        "database": "typescript_crud_api"
    },
    "jwtSecret": "your_jwt_secret_key"
}
```

Initialization Sequence

- Load configuration from JSON
- Establish MySQL connection
- Create database if it does not exist
- Initialize Sequelize instance
- Sync all registered models

Users Table Schema

```
CREATE TABLE users (
    id           INT          AUTO_INCREMENT PRIMARY KEY,
    email        VARCHAR(255) UNIQUE NOT NULL,
    passwordHash VARCHAR(255) NOT NULL,
    title        VARCHAR(100) NOT NULL,
    firstName    VARCHAR(100) NOT NULL,
    lastName     VARCHAR(100) NOT NULL,
    role         VARCHAR(50)  NOT NULL,
    createdAt    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    updatedAt    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
                             ON UPDATE CURRENT_TIMESTAMP
);
```

4. Request Flow (POST /users)

A user-creation request passes through the following stages:

Step	Stage	Detail
1	Incoming Request	POST /users with JSON body
2	Route Matching	Express maps path to controller method
3	Validation	Required fields, email format, password rules
4	Controller	Calls userService.create(req.body)
5	Service Layer	Checks duplicate email, hashes password, inserts record
6	ORM Execution	Sequelize executes parameterised SQL INSERT
7	Response	200 OK · { "message": "User created successfully" }

Controller Pattern

```
function create(req: Request, res: Response, next: NextFunction): void {
    userService.create(req.body)
    .then(() => res.json({message: 'User created successfully'}))
    .catch(next);
}
```

5. Middleware Pipeline

Every request passes through the following middleware in order:

```
Request
   |
JSON Parser          parse req.body as JSON
   |
CORS                 set cross-origin headers
   |
Validation           schema checks on req.body
   |
Controller           route-level handler
   |
Service Layer        business logic
   |
Error Handler        catches all thrown errors
   |
Response
```

6. Error Handling

All errors are routed to a single four-argument Express middleware, ensuring a consistent error response shape across the entire API.

```
import type {Request, Response, NextFunction} from 'express';

export function errorHandler(
    err: Error | string,
    req: Request,
    res: Response,
    next: NextFunction
): Response | void {

    if(typeof err === 'string'){
        const is404 = err.toLowerCase().endsWith('not found');
        const statusCode = is404 ? 404 : 400;
        return res.status(statusCode).json({message: err});
    }

    if (err instanceof Error){
        return res.status(500).json({message: err.message});
    }

    return res.status(500).json({message: 'Internal Server Error'});
}
```

Status Code Reference

Condition	Status Code
String error ending in 'not found'	404 Not Found
Any other string error	400 Bad Request
Error instance	500 Internal Server Error
Unknown error type	500 Internal Server Error

7. Authentication & Security

Password Hashing

```
const passwordHash = await bcrypt.hash(params.password, 10);
```

```
const user = await db.User.scope('withHash').findByPk(id);
```

Security Practices

- **No plaintext passwords** – all passwords are hashed with bcrypt before storage
- **Response pruning** – passwordHash is excluded from every API response
- **Unique email constraint** – enforced at the database level to prevent duplicates
- **Cost factor 10** – balances security against acceptable response latency

8. TypeScript Configuration

```
{
  "compilerOptions": {
    "target": "es2020",
    "module": "commonjs",
    "lib": ["ES2020"],
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "baseUrl": "./src",
    "paths": {
      "_helpers/*" : ["_helpers/*"],
      "_middleware/*" : ["_middleware/*"],
      "users/*" : ["users/w"]
    }
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Option	Purpose
strict	Enables all strict type checks — catches bugs at compile time
esModuleInterop	Allows default imports from CommonJS modules
resolveJsonModule	Allows direct import of .json config files
baseUrl	Enables path aliases instead of deep relative imports
ES2020 target	Unlocks optional chaining, nullish coalescing, and more

9. Troubleshooting

Database Connection Failure

- Confirm the MySQL service is running
- Validate credentials in the config file
- Check that port 3306 is not blocked

Port Already in Use

```
# Find the process occupying port 4000
netstat -ano | findstr :4000

# Kill the process
taskkill /PID <PID> /F
```

Type Errors — Role

```
// Incorrect — raw string bypasses type safety
role: "Admin"

// Correct — use the enum
role: Role.Admin
```

Module Not Found

- Check baseUrl and paths in tsconfig.json
- Ensure the directory structure matches the configured aliases

10. Performance Considerations

Area	Consideration
Email Index	Indexed column allows fast lookup without full table scans
bcrypt cost = 10	Tuned to ~100 ms hash time — strong security, low overhead
Field Pruning	Excluding passwordHash reduces response payload size
Early Validation	Malformed requests are rejected before reaching the database

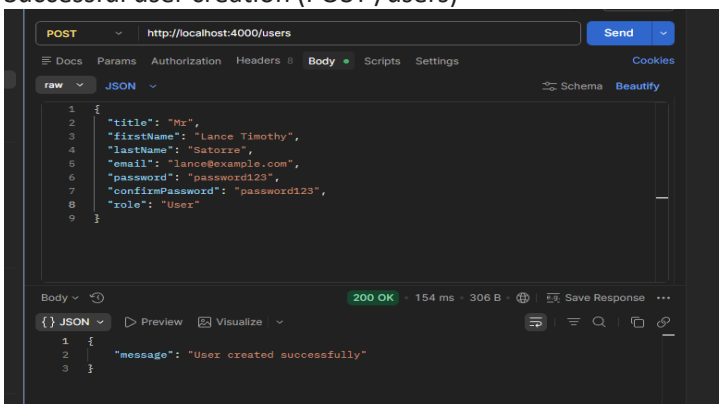
11. Future Enhancements

- JWT-based authentication
- Role-Based Access Control (RBAC)
- Pagination and filtering
- Email verification on registration
- Automated testing with Jest and Supertest

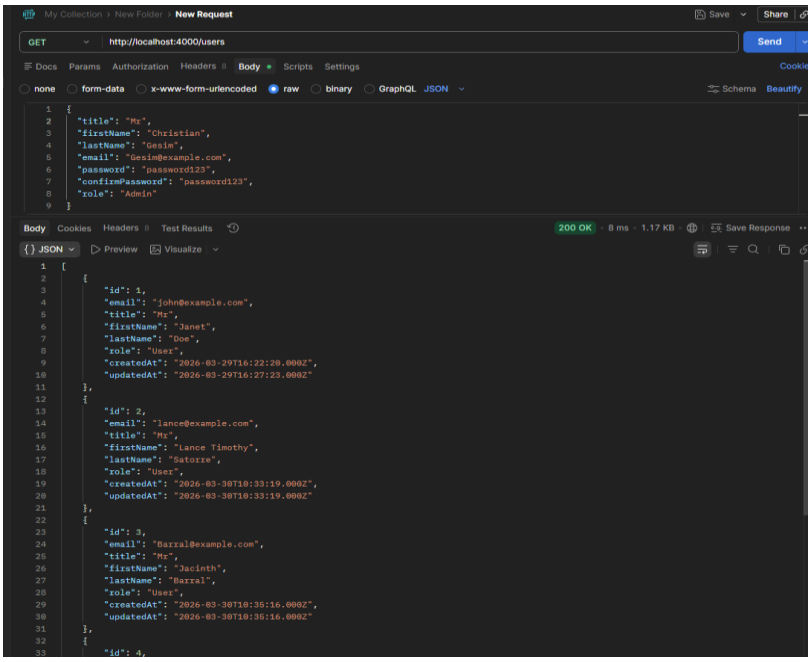
Deliverables

- **API Test Evidence**

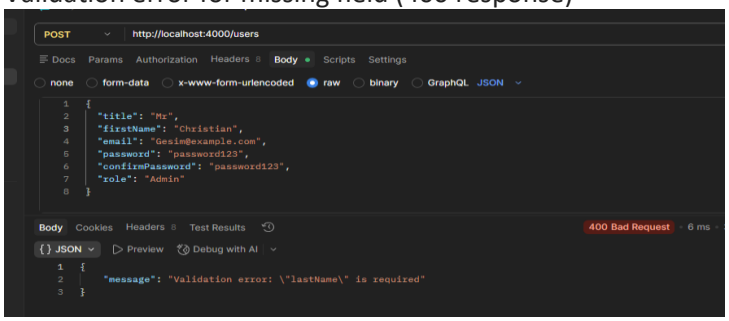
  - Successful user creation (POST /users)



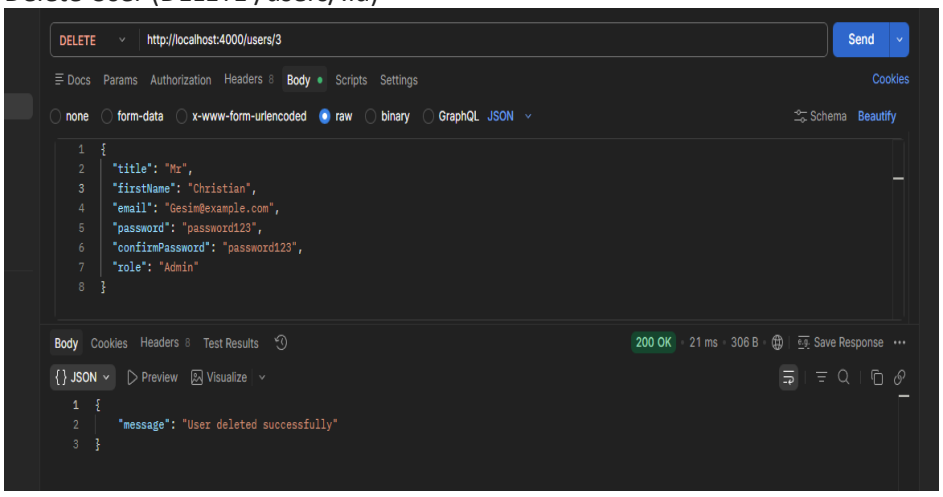
- Get all users (GET /users) showing typed response



- Validation error for missing field (400 response)



- Delete User (DELETE /users/:id)



- **Short Reflection (3-5 sentences):**
  "How did TypeScript change your development experience? Did you catch any errors at compile time that would have been runtime bugs in JavaScript?

  - TypeScript significantly improved my development experience by catching type-related errors before runtime. For example, the strongly-typed User model interfaces (`UserAttributes`, `UserCreationAttributes`) prevented accidental property access on undefined fields. The `Role` enum ensured only valid role values ('Admin', 'User') could be assigned, eliminating invalid string bugs that would have occurred in plain JavaScript. The strict function signatures (return types like `Promise<User[]>`) made it clear what each service method returns, reducing debugging time. Overall, TypeScript transformed potential runtime headaches into compile-time catches, making the codebase more maintainable and reliable.

Summary

This API follows a clean layered architecture, providing a clear separation of concerns across every component. Each layer has a single, well-defined responsibility, making the codebase maintainable, scalable, and straightforward to extend.

- **Maintainable** – layered structure makes changes local and predictable
- **Scalable** – stateless design supports horizontal scaling
- **Secure** – bcrypt hashing, field pruning, and DB-level constraints
- **Type safe** – TypeScript strict mode enforced end-to-end